3 - Representação de Dados

Quando você viaja, pode usar um mapa rodoviário – seja o antigo, desdobrável, ou uma versão eletrônica apresentada por um sistema de navegação. Seja qual for o formato, o mapa não é a terra sobre a qual você viaja, mas sim uma representação dessa terra. O mapa capturou algumas das informações necessárias para atingir o objetivo de ir de um lugar para outro.

Da mesma forma, os dados que precisamos armazenar e gerenciar em um computador devem ser representados de uma forma que capture a essência das informações com as quais nos importamos, e devem fazê-lo em um formato conveniente para o processamento do computador. Com base nos conceitos fundamentais do sistema numérico binário estabelecidos no Capítulo 2, este capítulo explora como representamos e armazenamos os vários tipos de dados que um computador gerencia.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- distinguir entre dados analógicos e digitais.
- explicar a compactação de dados e calcular taxas de compactação.
- explicar os formatos binários para valores negativos e de ponto flutuante.
- descrever as características dos conjuntos de caracteres ASCII e Unicode.
- realizar vários tipos de compressão de texto.
- explicar a natureza do som e sua representação.
- explicar como os valores RGB definem uma cor.
- distinguir entre gráficos *raster* e vetoriais.
- explicar a compressão de vídeo temporal e espacial.

3.1 Dados e Computadores

Sem dados, os computadores seriam inúteis. Toda tarefa que um computador realiza envolve gerenciar dados de alguma forma. Portanto, nossa necessidade de representar e organizar dados de maneiras apropriadas é primordial.

Vamos começar distinguindo os termos dados e informação. Embora esses termos sejam frequentemente usados de forma intercambiável, fazer a distinção às vezes é útil, especialmente em computação. Dados são valores ou fatos básicos, enquanto informação são dados que foram organizados e/ou processados de forma útil para resolver algum tipo de problema. Dados podem ser desestruturados e carecer de contexto. A informação nos ajuda a responder perguntas (ela "informa"). Essa distinção, é claro, é relativa às necessidades do usuário, mas captura a essência do papel que os computadores desempenham em nos ajudar a resolver problemas.

* Dados: Valores ou fatos básicos

* Informação: Dados que foram organizados ou processados de maneira útil

Neste capítulo, focamos na representação de diferentes tipos de dados. Em capítulos posteriores, discutiremos as várias maneiras de organizar dados para resolver tipos específicos de problemas.

Em um passado não tão distante, os computadores lidavam quase exclusivamente com dados numéricos e textuais. Hoje, no entanto, os computadores são verdadeiros dispositivos multimídia, lidando com uma vasta gama de categorias de informação. Os computadores armazenam, apresentam e nos ajudam a modificar muitos tipos diferentes de dados:

- Números
- Texto
- Áudio
- Imagens e gráficos
- Vídeo

* Multimídia: Vários tipos diferentes de mídia

Em última análise, todos esses dados são armazenados como dígitos binários. Cada documento, imagem e trecho de áudio é, de alguma forma, representado como sequências de 0s e 1s. Este capítulo explora cada um desses tipos de dados separadamente e discute as ideias básicas por trás das maneiras como representamos esses tipos de dados em um computador.

Não podemos discutir a representação de dados sem também falar sobre compactação de dados – reduzindo a quantidade de espaço necessária para armazenar um dado. No passado, precisávamos manter os dados em tamanho reduzido devido às limitações de armazenamento. Hoje, o armazenamento em computadores é relativamente barato – mas agora temos um motivo ainda mais urgente para reduzir nossos dados: a necessidade de compartilhá-los com outras pessoas. A web e suas redes subjacentes têm restrições inerentes de largura de banda que definem o número máximo de bits ou bytes que podem ser transmitidos de um lugar para outro em um período de tempo fixo.

A taxa de compressão indica o nível de compressão. A taxa de compressão é o tamanho dos dados compactados dividido pelo tamanho dos dados originais. Os valores podem estar em bits ou caracteres (ou o que for apropriado), desde que ambos os valores medirem a mesma coisa. A taxa deve resultar em um número entre 0 e 1. Quanto mais próxima a taxa estiver de zero, mais compacta será a compressão.

Uma técnica de compressão de dados pode ser sem perdas, o que significa que os dados podem ser recuperados sem perder nenhuma informação original, ou pode ser com perdas, caso em que algumas informações são perdidas no processo de compactação. Embora nunca queiramos perder informações, em alguns casos essa perda é aceitável. Ao lidar com representação e compressão de dados, sempre enfrentamos um dilema entre precisão e tamanho.

* Compressão de dados: Redução da quantidade de espaço necessária para armazenar um dado.

* Largura de banda: Número de bits ou bytes que podem ser transmitidos de um lugar para outro em um período fixo de tempo.

* Taxa de compressão: O tamanho dos dados comprimidos dividido pelo tamanho dos dados descomprimidos.

* Compressão sem perdas: Uma técnica de compressão de dados na qual não há perda de informação.

* Compressão com perdas: Uma técnica de compressão de dados na qual há perda de informação.

Dados Analógicos e Digitais

O mundo natural, em sua maior parte, é contínuo e infinito. Uma reta numérica é contínua, com valores crescendo infinitamente maiores e menores. Ou seja, você sempre pode chegar a um número maior ou menor do que qualquer número dado. Da mesma forma, o espaço numérico entre dois inteiros é infinito. Por exemplo, qualquer número pode ser dividido ao meio, em terços, em quartos e etc. Mas o mundo não é apenas infinito no sentido matemático. O espectro de cores é um arco-íris contínuo de tons infinitos. Objetos no mundo real se movem por um espaço contínuo e infinito. Teoricamente, você sempre poderia reduzir a distância entre você e uma parede pela metade, e nunca chegaria de fato à parede.

Os computadores, por outro lado, são finitos. A memória do computador e outros dispositivos de hardware têm apenas um espaço limitado para armazenar e manipular uma certa quantidade de dados. Sempre falhamos em nossa tentativa de representar um mundo infinito em uma máquina finita. O objetivo, então, é representar o suficiente do mundo para satisfazer nossas necessidades computacionais e nossos sentidos de visão e audição. Queremos tornar nossas representações boas o suficiente para realizar a tarefa, qualquer que seja ela.

Os dados podem ser representados de duas maneiras: analógica ou digital. Dados analógicos são uma representação contínua, análoga à informação real que representam. Dados digitais são uma representação discreta, dividindo a informação em elementos separados.

* Dados analógicos: Uma representação contínua de dados

* Dados digitais: Uma representação discreta de dados

Um termômetro de mercúrio é um dispositivo analógico. O mercúrio sobe em um fluxo contínuo no tubo em proporção direta à temperatura. Calibramos e marcamos o tubo para que possamos ler a temperatura atual, geralmente como um número inteiro, como 35 graus Celsius. No entanto, o mercúrio nesse termômetro está, na verdade, subindo continuamente entre os graus. Em algum momento, a temperatura é, na verdade, 35,568 graus Celsius, e o mercúrio está indicando isso com precisão, mesmo que nossas marcações não sejam detalhadas o suficiente para notar essas pequenas mudanças. Veja a FIGURA 3.1.

Dados analógicos são diretamente proporcionais ao mundo contínuo e infinito ao nosso redor. Computadores, portanto, não funcionam bem com dados analógicos. Em vez disso, digitalizamos dados dividindo-os em partes e representando essas partes separadamente. Cada uma das representações discutidas neste capítulo pega uma entidade contínua e a separa em elementos discretos. Esses elementos discretos são então representados individualmente usando dígitos binários.

Mas por que usamos o sistema binário? Sabemos pelo Capítulo 2 que o binário é apenas um dos muitos sistemas numéricos equivalentes. Não poderíamos usar, digamos, o sistema numérico decimal, com o qual já estamos mais familiarizados? Poderíamos. De fato, isso já foi feito. Computadores foram construídos com base em outros sistemas numéricos. No entanto, os computadores modernos são projetados para usar e gerenciar valores binários porque os dispositivos que armazenam e gerenciam os dados são muito mais baratos e muito mais confiáveis se tiverem que representar apenas um de dois valores possíveis.

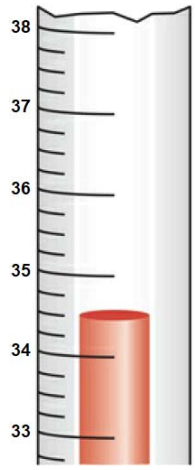


FIGURA 3.1 Um termômetro de mercúrio sobe continuamente em proporção direta à temperatura.

* Digitalizar: O ato de dividir informações em partes discretas.

Além disso, sinais eletrônicos são muito mais fáceis de manter se transferirem apenas dados binários. Um sinal analógico oscila continuamente para cima e para baixo em tensão, mas um sinal digital tem apenas um estado alto ou baixo, correspondente aos dois dígitos binários. Veja a FIGURA 3.2.

Todos os sinais eletrônicos (analógicos e digitais) se degradam à medida que se movem ao longo de uma linha. Ou seja, a tensão do sinal flutua devido aos efeitos ambientais. O problema é que, assim que um sinal analógico se degrada, a informação é perdida. Como qualquer nível de tensão dentro da faixa é válido, é impossível saber qual era o estado original do sinal ou mesmo se ele mudou.

Os sinais digitais, por outro lado, oscilam bruscamente entre dois extremos – um comportamento conhecido como modulação por código de pulso (PCM – *pulse-code modulation*). Um sinal digital pode se degradar bastante antes que qualquer informação seja perdida, porque qualquer valor de tensão acima de um determinado limite é considerado um valor alto, e qualquer valor abaixo desse limite é considerado um valor baixo. Periodicamente, um sinal digital é reclockado para recuperar sua forma original. Desde que seja reclockado antes que ocorra muita degradação, nenhuma informação é perdida. A FIGURA 3.3 mostra os efeitos da degradação de sinais analógicos e digitais.



FIGURA 3.2 Um sinal analógico e um sinal digital.

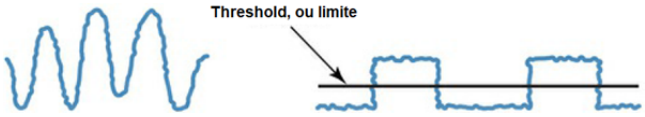


FIGURA 3.3 Degradação de sinais analógicos e digitais.

* Modulação: por código de pulso Variação em um sinal que salta bruscamente entre dois extremos.

* Reclockar: Ato de reafirmar um sinal digital original antes que ocorra muita degradação. Reclockado significa que o sinal foi regenerado e sincronizado com um novo relógio, ou seja, foi corrigido e otimizado para garantir sua integridade e precisão.

Representações Binárias

Ao investigarmos os detalhes da representação de tipos específicos de dados, é importante lembrar a natureza inerente do uso do binário. Um bit pode ser 0 ou 1. Não existe ausência de valor e nem uma terceira possibilidade. Portanto, um bit pode representar apenas duas coisas. Por exemplo, se quiséssemos classificar um alimento como doce ou azedo, precisaríamos de apenas um bit para isso. Poderíamos dizer que, se o bit for 0, o alimento é doce e, se o bit for 1, o alimento é azedo. Mas se quisermos classificações adicionais (como picante), um bit não é suficiente.

Para representar mais de duas coisas, precisamos de vários bits. Dois bits podem representar quatro coisas porque quatro combinações de 0 e 1 podem ser feitas a partir de dois bits: 00, 01, 10 e 11. Por

exemplo, se quisermos representar em qual das quatro marchas possíveis um carro está (estacionamento, direção, ré ou ponto morto), precisamos apenas de dois bits: estacionamento pode ser representado por 00, direção por 01, ré por 10 e ponto morto por 11. O mapeamento real entre as combinações de bits e o que cada combinação representa às vezes é irrelevante (00 pode ser usado para representar ré, se preferir), embora às vezes o mapeamento possa ser significativo e importante, como discutiremos em seções posteriores deste capítulo.

Se quisermos representar mais de quatro coisas, precisamos de mais de dois bits. Por exemplo, três bits podem representar oito possibilidades porque oito combinações de 0 e 1 podem ser feitas a partir de três bits. Da mesma forma, quatro bits podem representar 16 possibilidades, cinco bits podem representar 32 possibilidades e assim por diante. Na FIGURA 3.4, observe que as combinações de bits são simplesmente contadas em binário à medida que você desce uma coluna.

Em geral, n bits podem representar 2^n possibilidades porque 2^n combinações de 0 e 1 podem ser feitas a partir de n bits. Cada vez que aumentamos o número de bits disponíveis em 1, dobramos o número de combinações que podemos representar.

Vamos inverter a questão. Quantos bits você precisa para representar, digamos, 25 combinações únicas? Bem, quatro bits não seriam suficientes porque quatro bits podem representar apenas 16 coisas. Teríamos que usar pelo menos cinco bits, o que nos permitiria representar 32 combinações.

1 Bit	2 Bits	3 Bits	4 Bits
0	00	000	0000
1	01	001	0001
	10	010	0010
	11	011	0011
		100	0100
		101	0101
		110	0110
		111	0111
			1000
			1001
			1010
			1011
			1100
			1101
			1110
			1111

FIGURA 3.4 Combinações de bits.

Dado que precisamos representar apenas 25 combinações, algumas das combinações de bits não teriam uma interpretação válida.

Tenha em mente que, embora tecnicamente possamos precisar apenas de um certo número mínimo de bits para representar um conjunto de itens, podemos alocar mais do que isso para o armazenamento de dados. Há um número mínimo de bits que uma arquitetura de computador pode endereçar e transferir simultaneamente, e geralmente é uma potência de 2, como 8, 16 ou 32 bits. Portanto, a quantidade mínima de armazenamento fornecida a qualquer tipo de dado é alocada em múltiplos desse valor.

3.2 Representando Dados Numéricos

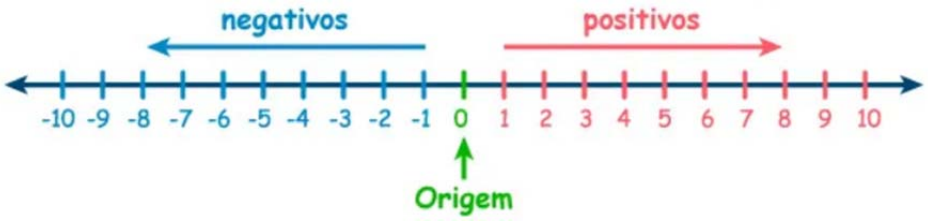
Valores numéricos são o tipo de dado mais prevalente usado em um sistema computacional. Ao contrário de outros tipos de dados, pode parecer que não há necessidade de criar um mapeamento inteligente entre códigos binários e dados numéricos. Como o binário é um sistema numérico, existe uma relação natural entre os dados numéricos e os valores binários que armazenamos para representá-los. Isso é verdade, em geral, para dados inteiros positivos. As questões básicas sobre conversões de números inteiros foram abordadas no Capítulo 2, na discussão geral sobre o sistema binário e sua equivalência com outras bases. No entanto, temos outras questões sobre a representação de dados numéricos a considerar neste ponto. Os números inteiros são apenas o começo em termos de dados numéricos. Esta seção discute a representação de valores negativos e não inteiros.

Representando Valores Negativos

Números negativos não são apenas números com um sinal de menos na frente? Talvez. Essa é certamente uma maneira válida de pensar sobre eles. Vamos explorar a questão dos números negativos e discutir maneiras apropriadas de representá-los em um computador.

Representação em Magnitude com Sinal

Você usa a representação de magnitude com sinal de números desde que aprendeu sobre números negativos no ensino fundamental. No sistema decimal tradicional, um sinal (+ ou –) é colocado antes do valor de um número, embora o sinal positivo seja frequentemente assumido. O sinal representa a ordenação e os dígitos representam a magnitude do número. A reta numérica clássica se parece com isto, na qual um sinal negativo significa que o número está à esquerda de zero e o número positivo está à direita de zero:



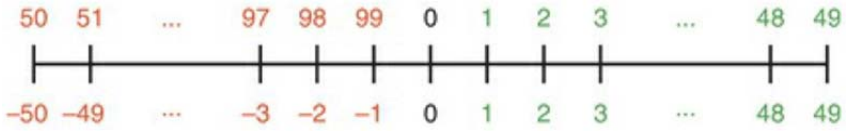
* Representação de magnitude com sinal: Representação numérica na qual o sinal representa a ordenação do número (negativo e positivo) e o valor representa a magnitude.

A adição e a subtração com números inteiros com sinal podem ser descritas como mover um certo número de unidades em uma direção ou outra. Para somar dois números, você encontra o primeiro número na escala e move na direção do sinal do segundo quantas unidades forem especificadas. A subtração é feita de maneira semelhante, movendo-se ao longo da reta numérica conforme ditado pelo sinal e pela operação. No ensino fundamental, você logo aprendeu a fazer adição e subtração sem usar a reta numérica.

Há um problema com a representação de magnitude com sinal: existem duas representações de zero – o zero positivo e o zero negativo. A ideia de um zero negativo não necessariamente nos incomoda; nós simplesmente a ignoramos. No entanto, duas representações de zero em um computador podem causar complexidade desnecessária, então outras representações de números negativos são usadas. Vamos examinar outra alternativa.

Números de Tamanho Fixo

Se permitirmos apenas um número fixo de valores, podemos representar números apenas como valores inteiros, onde metade deles representa números negativos. O sinal é determinado pela magnitude do número. Por exemplo, se o número máximo de dígitos decimais que podemos representar for dois, podemos considerar de 1 a 49 serem os números positivos de 1 a 49 e considerar de 50 a 99 representar os números negativos de –50 a –1. Vamos pegar a reta numérica e numerar os valores negativos no topo de acordo com este esquema:



Para realizar a adição dentro deste esquema, basta somar os números e descartar qualquer transporte. Adicionar números positivos deve ser aceitável. Vamos tentar somar um número positivo e um número negativo, um número negativo e um número positivo, e dois números negativos. Eles são mostrados na tabela a seguir em magnitude com sinal e neste esquema (os carregamentos são descartados):

Sinal Magnitude	Novo Esquema
5	5
+ – 6	+ 94
– 1	99
– 4	96
+ 6	+ 6
2	2
– 2	98
+ – 4	+ 96
– 6	94

E quanto à subtração, usando este esquema para representar números negativos? A chave está na relação entre adição e subtração: A – B = A + (–B). Podemos subtrair um número de outro somando o negativo do segundo ao primeiro:

Sinal Magnitude	Novo Esquema		Adição Negativa
– 5	95		95
– 3	– 3	→	+ 97
– 8			92

Neste exemplo, assumimos um tamanho fixo de 100 valores e mantivemos nossos números pequenos o suficiente para usar a reta numérica para calcular a representação negativa de um número. No entanto, você também pode usar uma fórmula para calcular a representação negativa:

$$\texttt{Negativo(I)} = 10^{k} - \texttt{I, onde k é o número de dígitos.}$$

Vamos aplicar esta fórmula para –3 em nossa representação de dois dígitos:

$$-(3) = 10^2 - 3 = 97$$

E quanto a uma representação de três dígitos?

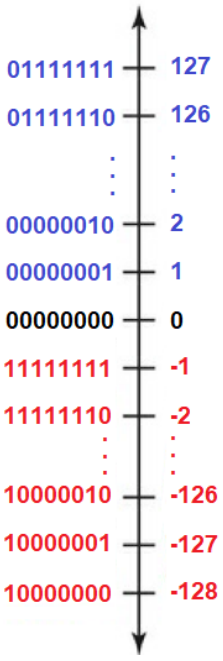
$$-(3) = 10^3 - 3 = 997$$

Essa representação de números negativos é chamada de complemento de dez. Embora os humanos tenham a pensar em termos de sinal e magnitude para representar números, a estratégia do complemento é, na verdade, mais fácil em alguns aspectos quando se trata de cálculos eletrônicos. Como armazenamos tudo em um computador moderno em binário, usamos o equivalente binário do complemento de dez chamado de **complemento de dois**.

* Complemento de dez: Uma representação de números negativos tal que o negativo de I é 10 elevado a k menos I.

Complemento de dois

Vamos supor que um número deva ser representado em oito bits, sete para o número e um para o sinal. Para facilitar a visualização de números binários longos, tornamos a reta numérica vertical, conforme figura ao lado.



A fórmula do complemento de dez funcionaria com o 10 substituído por 2? Ou seja, poderíamos calcular a representação binária negativa de um número usando a fórmula $\texttt{negativo(I)} = 2^{k} - \texttt{I}$? Vamos tentar e ver:

$$-(2) = 2^7 - 2 = 128 - 2 = -126$$

O decimal 126 é o octal 176, que é 1111110 em binário – mas na reta numérica o valor –2 tem mais um dígito 1 à esquerda. O que há de errado? Nada. Este é um número negativo. O dígito mais à esquerda determina se o número é negativo ou positivo. Um bit 0 no dígito mais à esquerda diz que o número é positivo; um bit 1 diz que o número é negativo. Portanto, –(2) é 11111110.

Há uma maneira mais fácil de calcular o complemento de dois: inverta os bits do número e some 1. Ou seja, pegue o valor positivo e altere todos os bits 1 para 0 e todos os bits 0 para 1, e então some 1.

+2	= 00000010
Inverte bits	11111101
Soma 1	+ 00000001
–2	= 11111110

A adição e a subtração são realizadas da mesma forma que na aritmética do complemento de dez, conforme ao lado. Com esta representação, o bit mais à esquerda em um número negativo é sempre 1. Portanto, você pode dizer imediatamente se um número binário em complemento de dois é negativo ou positivo.

– 127	10000001
+ 1	00000001
– 126	10000010

Estouro de Número

O estouro (ou *overflow*) ocorre quando o valor que calculamos não cabe no número de bits que alocamos para o resultado. Por exemplo, se cada valor for armazenado usando oito bits, somar 127 + 3 produziria um estouro.

```
 1111111
 01111111
+00000011
 10000010
```

* Estouro (overflow): Uma situação em que um valor calculado não cabe no número de dígitos reservado para ele.

Em nosso esquema, o resultado 10000010 representa –126, não +130. Se não estivéssemos representando números negativos, no entanto, o resultado estaria correto.

O estouro é um exemplo clássico do tipo de problema que encontramos ao mapear um mundo infinito em uma máquina finita. Não importa quantos bits alocamos para um número, sempre há a necessidade potencial de representar um número que não cabe. A forma como os problemas de estouro são tratados varia de acordo com o hardware do computador e as diferenças nas linguagens de programação.

Representando Números Reais

Em computação, chamamos todos os valores não inteiros (que podem ser representados) de valores reais. Para nossos propósitos aqui, definimos um número real como um valor com uma parte fracionária potencial. Ou seja, números reais têm uma parte inteira e uma parte fracionária, sendo que qualquer uma delas pode ser zero. Por exemplo, alguns números reais na base 10 são 104.32, 0.9999, 357.0, 3.14159, -23.67 e -1.5.

Como exploramos no Capítulo 2, os dígitos representam valores de acordo com sua posição, e esses valores de posição são relativos à base. À esquerda do ponto decimal, na base 10, temos a posição das unidades, a posição das dezenas, a posição das centenas e assim por diante. Esses valores de posição vêm da elevação do valor da base a potências crescentes (movendo-se do ponto decimal para a esquerda). As posições à direita do ponto decimal funcionam da mesma maneira, exceto que as potências são negativas. Portanto, as posições à direita do ponto decimal são a posição dos décimos (10^{-1} ou um décimo), a posição dos centésimos (10^{-2} ou um centésimo), a posição dos milésimos (10^{-3} ou um milésimo) e assim por diante.

Em binário, as mesmas regras se aplicam, mas o valor da base é 2. Como não estamos trabalhando na base 10, o ponto decimal é chamado de ponto de base, um termo que pode ser usado em qualquer base. As posições à direita da vírgula de base em binário são a posição das metades (2^{-1} ou um meio), a posição dos quartos (2^{-2} ou um quarto), a posição dos oitavos (2^{-3} ou um oitavo) e assim por diante.

Como representamos um valor real em um computador? Armazenamos o valor como um inteiro e incluímos informações que mostram onde está o ponto de base. Ou seja, qualquer valor real pode ser descrito por três propriedades: o sinal (positivo ou negativo; a mantissa, que é composta pelos dígitos do valor com o ponto à direita; e o expoente, que determina como o ponto é deslocado em relação à mantissa. Um valor real na base 10 pode, portanto, ser definido pela seguinte fórmula:

$$\texttt{sinal} * \texttt{mantissa} * 10^{\texttt{exp}}$$

A representação é chamada de ponto flutuante porque o número de dígitos é fixo, mas o ponto flutua. Quando um valor está na forma de ponto flutuante, um expoente positivo desloca o ponto decimal para a direita e um expoente negativo desloca o ponto decimal para a esquerda.

* Ponto de base: O ponto que separa a parte inteira da parte fracionária em um número real em qualquer base.

* Ponto flutuante: Uma representação de um número real que acompanha o sinal, a mantissa e o expoente.

Vejamos como converter um número real expresso em nossa notação decimal usual para a notação de ponto flutuante. Como exemplo, considere o número 148.69. O sinal é positivo, e dois dígitos aparecem à direita do ponto decimal. Portanto, o expoente é –2, resultando em $14869 * 10^{-2}$.

A TABELA 3.1 mostra outros exemplos. Para fins desta discussão, assumimos que apenas cinco dígitos podem ser representados. Como convertemos um valor em forma de ponto flutuante de volta para a notação decimal? O expoente na base nos diz quantas posições mover o ponto decimal. Se o expoente for negativo, movemos o ponto decimal para a esquerda. Se o expoente for positivo, movemos o ponto decimal para a direita. Aplique este esquema aos valores de ponto flutuante na Tabela 3.1.

TABELA 3.1 Valores em notação decimal e em notação de ponto flutuante (com 5 dígitos).

Valor em Real	Valor em Ponto Flutuante
12001.00	$12001 * 10^{0}$
–120.01	$-12001 * 10^{-2}$
0.12000	$12000 * 10^{-5}$
–123.10	$-12310 * 10^{-2}$
155555000.00	$15555 * 10^{4}$

Observe que, no último exemplo da Tabela 3.1, perdemos informações. Como estamos armazenando apenas cinco dígitos para representar os dígitos significativos (a mantissa), a parte inteira do valor não é representada com precisão na notação de ponto flutuante.

Da mesma forma, um valor binário de ponto flutuante é definido pela seguinte fórmula:

$$sinal * mantissa * 2^{exp}$$

Observe que apenas o valor da base foi alterado. É claro que, neste esquema, a mantissa conteria apenas dígitos binários. Para armazenar um número de ponto flutuante em binário em um computador, podemos armazenar os três valores que o definem. Por exemplo, de acordo com um padrão comum, se dedicarmos 64 bits para o armazenamento de um valor de ponto flutuante, usaremos 1 bit para o sinal, 11 bits para o expoente e 52 bits para a mantissa. Internamente, esse formato é levado em consideração sempre que o valor é usado em um cálculo ou exibido.

Mas como obtemos o valor correto para a mantissa se o valor não for um número inteiro? No Capítulo 2, discutimos como alterar um número natural de uma base para outra. Aqui, mostramos como os números reais são representados em um computador, usando exemplos decimais. Sabemos que todos os valores são representados em binário no computador. Como transformamos a parte fracionária de um valor decimal em binário?

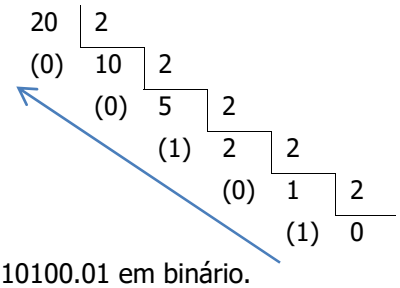
Para converter um valor inteiro da base 10 para outra base, dividimos pela nova base, registrando o resto como o próximo dígito à esquerda no resultado e continuando a dividir o quociente pela nova base até que o quociente se torne zero. A conversão da parte fracionária é semelhante, mas multiplicamos pela nova base em vez de dividir. O transporte da multiplicação se torna o próximo dígito à direita na resposta. A parte fracionária do resultado é então multiplicada pela nova base. O processo continua até que a parte fracionária do resultado seja zero. Vamos converter 0.75 para binário.

```
0.75 * 2 = 1.50     parte inteira é 1, toma a parte decimal .50
0.50 * 2 = 1.00     parte inteira é 1, e a parte decimal igual a zero.
```

Portanto, 0.75 em decimal é 0.11 em binário. Vamos tentar outro exemplo, o número 0.435.

```
0.435 * 2 = 0.870   parte inteira é 0, toma a parte decimal .870
0.870 * 2 = 1.740   parte inteira é 1, toma a parte decimal .740
0.740 * 2 = 1.480   parte inteira é 1, toma a parte decimal .480
0.480 * 2 = 0.960   parte inteira é 0, toma a parte decimal .960
0.960 * 2 = 1.920   parte inteira é 1, toma a parte decimal .920
0.920 * 2 = 1.840   parte inteira é 1, toma a parte decimal .840
. . .
```

Portanto, 0.435 é 011011… em binário. A parte fracionária algum dia se tornará zero? Continue multiplicando e veja.



Agora, vamos percorrer todo o processo de conversão, convertendo 20.25 de decimal para binário. Primeiro, convertemos 20. Após o processo de divisão sucessiva, verificamos que 20 em binário é 10100. Depois convertemos a parte fracionária:

```
0.25 * 2 = 0.50
0.50 * 2 = 1.00     Portanto, 20.25 em decimal é 10100.01 em binário.
```

Notação científica é um termo com o qual você já deve estar familiarizado, por isso o mencionamos aqui. A notação científica é uma forma de representação em ponto flutuante na qual o ponto decimal é mantido à direita do dígito mais à esquerda. Ou seja, há um número inteiro. Em muitas linguagens de programação, se você imprimir um valor real grande sem especificar como imprimi-lo, o valor é impresso em notação científica. Como expoentes não podiam ser impressos em máquinas antigas, um "E" era usado em seu lugar. Por exemplo, o número "12001.32708" seria escrito como "1.200132708E+4" em notação científica.

* Notação científica: Uma representação alternativa em ponto flutuante.

3.3 Representando Texto

Um documento de texto pode ser decomposto em parágrafos, frases, palavras e, por fim, caracteres individuais. Para representar um documento de texto em formato digital, precisamos simplesmente ser capazes de representar todos os caracteres possíveis que possam aparecer. O documento é a entidade contínua (analógica), e os caracteres separados são os elementos discretos (digitais) que precisamos representar e armazenar na memória do computador.

Neste ponto, devemos distinguir entre a ideia básica de representar texto e o conceito mais complexo de processamento de texto. Quando criamos um documento em um programa de processamento de texto

como o Microsoft® Word, podemos especificar todos os tipos de formatação: fontes, margens, tabulações, cores e assim por diante. Muitos processadores de texto também nos permitem adicionar arte, equações e outros elementos. Essas informações extras são armazenadas junto com o restante do texto para que o documento possa ser exibido e impresso da maneira que você desejar. A questão central, no entanto, é a maneira como representamos os próprios caracteres; portanto, essas técnicas continuam sendo nosso foco neste ponto.

Há um número finito de caracteres para representar. A abordagem geral para representar caracteres é listar todos eles e, em seguida, atribuir uma sequência binária a cada caractere. Por exemplo, para armazenar uma letra específica, armazenamos a sequência de bits apropriada.

Então, quais caracteres precisamos representar? O idioma inglês inclui 26 letras. Mas letras maiúsculas e minúsculas precisam ser tratadas separadamente, o que significa, na verdade, 52 caracteres únicos. Os caracteres de pontuação também precisam ser representados, assim como os dígitos numéricos (os caracteres reais de '0', '1' a '9'). Até mesmo os caracteres de espaço precisam ter uma representação. E quanto a outros idiomas além do inglês? A lista de caracteres que podemos querer representar começa a crescer rapidamente quando você começa a pensar sobre isso. Lembre-se de que, como discutimos anteriormente neste capítulo, o número de coisas únicas (caracteres, neste caso) que queremos representar determina quantos bits precisaremos para representar qualquer uma delas.

Um conjunto de caracteres é simplesmente uma lista de caracteres e os códigos usados para representar cada um. Vários conjuntos de caracteres foram usados ao longo dos anos, embora alguns tenham dominado. Ao concordar em usar um conjunto de caracteres específico, os fabricantes de computadores tornaram o processamento de dados de texto mais fácil. Exploraremos dois conjuntos de caracteres nas seguintes seções: ASCII e Unicode.

O Conjunto de Caracteres ASCII

O American Standard Code for Information Interchange (ASCII) – significa Código Padrão Americano para Intercâmbio de Informações. O conjunto de caracteres ASCII originalmente usava sete bits para representar cada caractere, permitindo 128 caracteres únicos. O oitavo bit em cada byte de caractere era originalmente usado como um bit de verificação, o que ajudava a garantir a transmissão adequada de dados. Posteriormente, o ASCII evoluiu para que todos os oito bits fossem usados para representar um caractere. Essa versão de oito bits é formalmente chamada de conjunto de caracteres ASCII Estendido Latin-1 (ou Latin-1 Extended ASCII). O conjunto ASCII estendido permite 256 caracteres e inclui letras acentuadas bem como vários outros símbolos especiais. A FIGURA 3.5 mostra o conjunto de caracteres ASCII.

Dígito direito / Dígito(s) esquerdo	**ASCII**									
	0	1	2	3	4	5	6	7	8	9
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT
1	LF	VT	FF	CR	SO	SI	DLE	DC1	DC2	DC3
2	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS
3	RS	US	□	!	"	#	$	%	&	´
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	L	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	DEL		

FIGURA 3.5 O conjunto de caracteres ASCII.

Os códigos na Figura 3.5 são expressos como números decimais, mas esses valores são traduzidos para seu equivalente binário para armazenamento no computador. Observe que os caracteres ASCII têm uma ordem distinta com base nos códigos usados para armazená-los. Cada caractere tem uma posição relativa (antes ou depois) de cada caractere. Essa propriedade é útil de várias maneiras diferentes. Por exemplo, observe que tanto as letras maiúsculas quanto as minúsculas estão em ordem. Portanto, podemos usar os códigos de caracteres para nos ajudar a colocar uma lista de palavras em ordem alfabética.

Os primeiros 32 caracteres na tabela de caracteres ASCII não têm uma representação simples de caracteres que você possa imprimir na tela. Esses caracteres são reservados para fins especiais, como retorno de carro, tabulação, etc. Eles geralmente são interpretados de maneiras especiais por qualquer programa que esteja processando os dados.

? Labirinto de conjuntos de caracteres

Em 1960, um artigo na Communications of the ACM relatou uma pesquisa sobre conjuntos de caracteres em uso. 60 conjuntos distintos foram descritos. Nove conjuntos de caracteres, com diferenças tanto no conteúdo quanto na ordem, existiam na linha de computadores da IBM.[1]

O Conjunto de Caracteres Unicode

A versão estendida do conjunto de caracteres ASCII fornece 256 caracteres, o que é suficiente para o inglês, mas não o suficiente para uso internacional. Essa limitação deu origem ao conjunto de caracteres Unicode, que tem uma influência internacional muito mais forte.

O objetivo dos criadores do Unicode é nada menos do que representar todos os caracteres em todos os idiomas usados no mundo inteiro, incluindo todos os ideogramas asiáticos. Ele também representa muitos caracteres de propósito especial, como símbolos científicos.O conjunto de caracteres Unicode é usado por muitas linguagens de programação e sistemas de computador atualmente. Em geral, a codificação usa 16 bits por caractere, mas é flexível, permitindo o uso de mais espaço por caractere, se necessário, para representar caracteres adicionais. Um aspecto conveniente do Unicode é que ele possui os caracteres ASCII como um subconjunto com os mesmos valores numéricos. A FIGURA 3.6 (ao lado) mostra alguns caracteres da parte não ASCII do conjunto de caracteres Unicode.

Code (Hex)	Character	Source
0041	A	English (Latin)
042F	Я	Russian (Cyrillic)
OE09	ก	Thai
13EA	Ꮺ	Cherokee
211E	℞	Letterlike Symbols
21CC	⇌	Arrows
282F	⠯	Braille
345F	佟	Chinese/Japanese/ Korean (Common)

FIGURA 3.6 Alguns caracteres no conjunto de caracteres Unicode.

Para manter a consistência, o Unicode foi projetado para ser um superconjunto do ASCII. Ou seja, os primeiros 256 caracteres no conjunto de caracteres Unicode correspondem exatamente ao conjunto de caracteres ASCII estendido incluindo os códigos usados para representa-los. Portanto, programas que assumem valores ASCII não são afetados, mesmo que o sistema subjacente adote a abordagem Unicode.

Compressão de Texto

Informações alfabéticas (texto) são um tipo fundamental de dado. Portanto, é importante encontrarmos maneiras de armazenar e transmitir texto de forma eficiente entre um computador e outro. As seções a seguir examinam três tipos de compressão de texto:

- Codificação por palavras-chave.
- Codificação por execução.
- Codificação Huffman.

Como discutiremos mais adiante neste capítulo, algumas das ideias subjacentes a essas técnicas de compressão de texto também entram em jogo ao lidar com outros tipos de dados.

Codificação de Palavras-Chave

Considere a frequência com que você usa palavras como "o", "e", "qual", "aquele" e "o quê". Se essas palavras ocupassem menos espaço (ou seja, tivessem menos caracteres), nossos documentos diminuiriam de tamanho. Embora a economia para cada palavra fosse pequena, elas são usadas com tanta frequência em um documento típico que a economia combinada aumentaria rapidamente.

Um método bastante simples de compactação de texto é a codificação por palavras-chave, na qual palavras usadas com frequência são substituídas por um único caractere. Para descompactar o documento, inverta o processo: substitua os caracteres individuais pela palavra completa apropriada.

Por exemplo, suponha que usamos a tabela ao lado para codificar algumas palavras.

Word	Symbol	Word	Symbol
as	^	must	&
the	~	well	%
and	+	these	#
that	$		

* Codificação por palavras-chave: substituição de uma palavra usada com frequência por um único caractere.

Vamos codificar o seguinte parágrafo:

> The human body is composed of many independent systems, such as the circulatory system, the respiratory system, and the reproductive system. Not only must all systems work independently, but they must interact and cooperate as well. Overall health is a function of the well-being of separate systems, as well as how these separate systems work in concert.

O parágrafo codificado é:

> The human body is composed of many independent systems, such ^ ~ circulatory system, ~ respiratory system, + ~ reproductive system. Not only & each system work independently, but they & interact + cooperate ^ %. Overall health is a function of ~ %-being of separate systems, ^ % ^ how # separate systems work in concert.

Há um total de 352 caracteres no parágrafo original, incluindo espaços e pontuação. O parágrafo codificado contém 317 caracteres, resultando em uma economia de 35 caracteres. A taxa de compressão para este exemplo é 317/352 ou aproximadamente 0,9.

Existem várias limitações na codificação de palavras-chave. Primeiro, observe que os caracteres que usamos para codificar as palavras-chave não podem fazer parte do texto original. Se, por exemplo, o "$" fizesse parte do parágrafo original, a codificação resultante seria ambígua. Não saberíamos se o "$" representava a palavra "that" ou se era o próprio caractere do cifrão. Isso limita o número de palavras que podemos codificar, bem como a natureza do texto que estamos codificando.

? Noite cara

Se você se hospedou em um hotel Holiday Inn, Holiday Inn Express ou Crowne Plaza e fez o check-out entre 24 e 26 de outubro de 2002, provavelmente foi uma das 26.000 pessoas que foram cobradas 100 vezes o que deviam, como de US$ 6.500 a US$ 21.000 por noite. Um erro de processamento de crédito resultou na eliminação das casas decimais.

Observe também que a palavra "The" no exemplo não é codificada pelo caractere '~' porque a palavra "The" e a palavra "the" contêm letras diferentes. Lembre-se de que as versões maiúsculas e minúsculas da mesma letra são caracteres diferentes quando se trata de armazená-las em um computador. Teríamos que usar um símbolo separado para "The" se quiséssemos codificá-lo, ou empregar um esquema de substituição mais sofisticado.

Finalmente, observe que não ganharíamos nada codificando palavras como "a" e "I" porque isso seria simplesmente substituir um caractere por outro. Quanto mais longa a palavra, mais compactação obtemos por palavra. Infelizmente, as palavras usadas com mais frequência são frequentemente curtas. É claro que alguns documentos usam certas palavras com mais frequência do que outras dependendo do assunto. Por exemplo, teríamos obtido uma boa economia se tivéssemos escolhido codificar a palavra "system" em nosso exemplo, mas pode não valer a pena codificar em uma situação geral.

Uma extensão da codificação de palavras-chave é substituir padrões específicos de texto por caracteres especiais. Os padrões codificados geralmente não são palavras completas, mas sim partes de palavras, como prefixos e sufixos comuns – "ex", "ing" e "tion", por exemplo. Uma vantagem dessa abordagem é que os padrões codificados geralmente aparecem com mais frequência do que palavras inteiras (porque ocorrem em muitas palavras diferentes). Uma desvantagem é que geralmente são padrões curtos e, portanto, a economia de substituição por palavra é pequena.

Codificação de Longa Duração

Em algumas situações, um único caractere pode ser repetido várias vezes em uma longa sequência. Esse tipo de repetição geralmente não ocorre em textos em inglês, mas frequentemente ocorre em grandes fluxos de dados, como sequências de DNA. Uma técnica de compressão de texto chamada codificação de longa duração tira proveito dessas situações. A codificação de longa duração às vezes é chamada de codificação de recorrência.

* Codificação de comprimento de execução: Substituição de uma longa série de um caractere repetido por uma contagem da repetição.

Na codificação de comprimento de execução, uma sequência de caracteres repetidos é substituída por um caractere sinalizador, seguido pelo caractere repetido, seguido por um único dígito que indica quantas vezes o caractere é repetido. Por exemplo, considere a seguinte sequência de sete caracteres 'A' repetidos:

AAAAAAA

Se usarmos o caractere '*' como sinalizador, esta sequência seria codificada como

*A7

O caractere sinalizador indica que a série de três caracteres (incluindo o sinalizador) deve ser decodificada na sequência repetitiva apropriada. Todo o restante do texto é tratado regularmente. Portanto, a sequência codificada

*n5*x9ccc*h6 algum outro texto *k8eee

seria decodificado no seguinte texto original:

nnnnnxxxxxxxxxccchhhhhh algum outro texto kkkkkkkkeee

O texto original contém 53 caracteres e a string codificada contém 37 caracteres, dando-nos uma taxa de compressão neste exemplo de 37/53, ou aproximadamente 0,69. Neste exemplo, os três caracteres 'c' repetidos e os três caracteres 'e' repetidos não são codificados. Como são necessários três caracteres para codificar uma sequência de repetições, não vale a pena codificar strings de dois ou três. De fato, no caso de dois caracteres repetidos, a codificação tornaria a string mais longa!

Dado que estamos usando um caractere para a contagem de repetições, parece que não podemos codificar comprimentos de repetição maiores que nove. Mas lembre-se de que cada caractere é representado por uma série de bits com base em algum conjunto de caracteres. Por exemplo, o caractere "5" é representado como valor ASCII 53, que em uma string binária de oito bits é 00110101. Portanto, em vez de interpretar o caractere de contagem como um dígito ASCII, poderíamos interpretá-lo como um número binário. Podemos então ter contagens de repetição entre 0 e 255, ou mesmo entre 4 e 259, porque séries de três ou menos não são representadas.

Codificação Huffman

Outra técnica de compressão de texto, chamada codificação Huffman, leva o nome de seu criador, Dr. David Huffman. Por que o caractere "X", raramente usado em texto, deveria ocupar o mesmo número de bits que o espaço em branco, que é usado com muita frequência? Os códigos Huffman resolvem essa questão usando sequências de bits de comprimento variável para representar cada caractere. Ou seja, alguns caracteres podem ser representados por cinco bits, outros por seis bits, outros ainda por sete bits e assim por diante. Essa abordagem é contrária à ideia de um conjunto de caracteres, no qual cada caractere é representado por uma sequência de bits de comprimento fixo (como 8 ou 16).

* Codificação Huffman: Usar uma sequência binária de comprimento variável para representar um caractere, de forma que os caracteres usados com frequência tenham códigos curtos.

A ideia por trás dessa abordagem é que, se usarmos apenas alguns bits para representar caracteres que aparecem com frequência e reservarmos sequências de bits mais longas para caracteres que não aparecem com frequência, o tamanho geral do documento representado será pequeno.

Por exemplo, suponha que usamos a codificação de Huffman da tabela ao lado para representar alguns caracteres.

Então, a palavra "DOORBELL" seria codificada em binário como

Código de Huffman	Caractere
00	A
01	E
100	L
110	O
111	R
1010	B
1011	D

10111101101111010011100100

Se usássemos uma sequência de bits de tamanho fixo para representar cada caractere (digamos, 8 bits), a forma binária da sequência original seria 8 caracteres vezes 8 bits, ou 64 bits. A codificação Huffman para

essa sequência tem 25 bits de comprimento, resultando em uma taxa de compressão de 25/64, ou aproximadamente 0,39.

E quanto ao processo de decodificação? Quando usamos conjuntos de caracteres, simplesmente pegamos os bits em pedaços de 8 ou 16 bits para ver qual caractere cada pedaço representa. Na codificação Huffman, com seus códigos de comprimento variável, parece que podemos ficar confusos ao tentar decodificar uma sequência de caracteres porque não sabemos quantos bits devemos incluir para cada caractere. Na verdade, essa potencial fonte de confusão foi eliminada pela forma como os códigos são criados.

Uma característica importante de qualquer codificação Huffman é que nenhuma sequência de bits usada para representar um caractere é o prefixo de qualquer outra sequência de bits usada para representar um caractere. Portanto, ao percorrermos uma sequência de bits da esquerda para a direita, quando encontramos uma sequência que corresponde a um caractere, esse deve ser o caractere que ela representa. Ela não pode fazer parte de uma sequência de bits maior.

Por exemplo, se a sequência de bits 1010110001111011 for criada com a tabela anterior, ela deve ser decodificada na palavra "BOARD". Não há outra possibilidade.

Então, como um conjunto específico de códigos de Huffman é criado? Os detalhes desse processo estão um pouco além do escopo deste livro, mas vamos discutir a questão subjacente. Como os códigos de Huffman usam as sequências de bits mais curtas para os caracteres mais comuns, começamos com uma tabela que lista a frequência dos caracteres que queremos codificar. As frequências podem vir da contagem de caracteres em um documento específico (352 E's, 248 S's e assim por diante) ou da contagem de caracteres em uma amostra de texto de uma área de conteúdo específica. Uma tabela de frequência também pode vir de uma ideia geral da frequência com que as letras ocorrem em um idioma específico como o inglês. Usando esses valores, podemos construir uma estrutura a partir da qual os códigos binários podem ser lidos. A maneira como a estrutura é criada garante que os caracteres mais usados recebam as sequências de bits mais curtas.

3.4 Representando Dados de Áudio

Percebemos o som quando uma série de compressões de ar vibra uma membrana em nosso ouvido, que envia sinais para o nosso cérebro. Assim, um som é definido na natureza pela onda de ar que interage com o nosso tímpano. Veja a FIGURA 3.7. Para representar um som, precisamos, de alguma forma, representar a onda sonora apropriada.

Um aparelho de som envia um sinal elétrico para um alto-falante para produzir som. Este sinal é uma representação analógica da onda sonora. A voltagem no sinal varia em proporção direta à onda sonora. O alto-falante recebe o sinal e faz uma membrana vibrar, que por sua vez vibra o ar (criando uma onda sonora), que por sua vez vibra o tímpano. A onda sonora criada deve, idealmente, ser idêntica à que foi capturada inicialmente, ou pelo menos boa o suficiente para agradar o ouvinte.

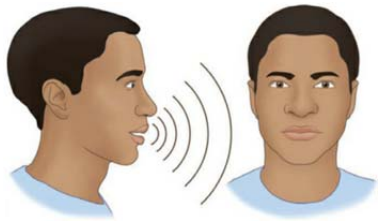


FIGURA 3.7 Uma onda sonora vibra nossos tímpanos.

Para representar dados de áudio em um computador, precisamos digitalizar a onda sonora, de alguma forma, dividindo-a em partes discretas e gerenciáveis. Uma maneira de realizar essa tarefa é digitalizar a representação analógica do som. Ou seja, podemos pegar o sinal elétrico que representa a onda sonora e representá-lo como uma série de valores numéricos discretos. Um sinal analógico varia em voltagem continuamente. Para digitalizar o sinal, medimos periodicamente a tensão do sinal e registramos o valor numérico apropriado. Esse processo é chamado de *sampling* (ou amostragem). Em vez de um sinal contínuo, obtemos uma série de números representando níveis de tensão distintos.

Para reproduzir o som, usamos os valores de tensão armazenados para criar um novo sinal eletrônico contínuo. A suposição é que os níveis de tensão no sinal original mudaram uniformemente entre um valor de tensão armazenado e o próximo. Se coletarmos amostras suficientes em um curto período de tempo, essa suposição é razoável. Mas certamente o processo de amostragem pode perder informações, como mostrado na FIGURA 3.8.

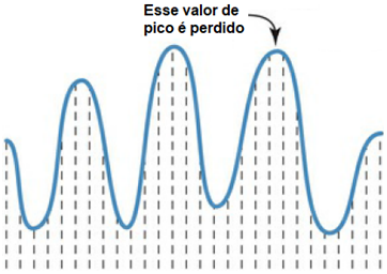


FIGURA 3.8 Amostragem de um sinal de áudio.

Em geral, uma taxa de amostragem de cerca de 40.000 vezes por segundo é suficiente para criar uma reprodução sonora razoável. Se a taxa de amostragem for muito menor que isso, o ouvido humano começa a ouvir distorções. Uma taxa de amostragem mais alta produz um som de melhor qualidade, mas, após um certo ponto, os dados extras são irrelevantes porque o ouvido humano não consegue ouvir a diferença. O resultado geral é afetado por muitos fatores, incluindo a qualidade do equipamento, o tipo de som e o ouvinte humano.

Um álbum de vinil é uma representação analógica da onda sonora. A agulha de um player (no caso, um toca-discos) sobe e desce na ranhura espiral do álbum. A subida e descida da agulha é análoga às variações de voltagem do sinal que representa o som.

Em contraste, um compact disk (CD) armazena informações de áudio digitalmente. Na superfície do CD existem cavidades microscópicas que representam dígitos binários. Um laser de baixa intensidade é apontado para o disco. A luz do laser reflete fortemente se a superfície for lisa e reflete mal se a superfície for perfurada. Um receptor analisa a reflexão e produz a sequência apropriada de dados binários, que representam os valores numéricos de voltagem que foram armazenados quando o sinal foi digitalizado. O sinal é reproduzido e enviado para o alto-falante. Veja a FIGURA 3.9.

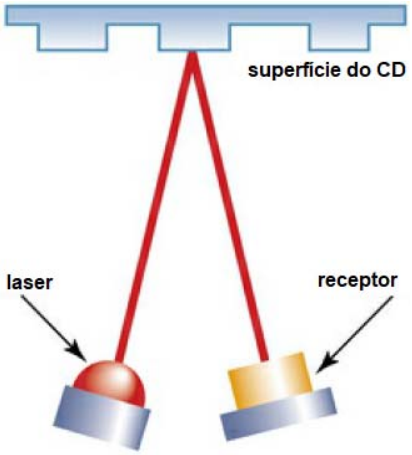


FIGURA 3.9 Um CD player lendo dados binários.

? Competição de atendimento telefônico no Japão

Por mais de cinquenta anos, trabalhadores de escritório em todo o Japão competiram pelo título de melhor atendente telefônico do Japão. As mulheres dominam a competição, usando as vozes agudas preferidas por muitos empresários japoneses. Elas são julgadas por suas maneiras e etiqueta empresarial, bem como por suas vozes. Será que os Estados Unidos poderiam ter uma competição para vozes digitais?

Formatos de Áudio

Nos últimos anos, diversos formatos para dados de áudio se tornaram populares, incluindo WAV, AU, AIFF, VQF e MP3. Todos esses formatos são baseados no armazenamento de valores de tensão amostrados de sinais analógicos, mas todos reconhecem os detalhes dos dados de maneiras diferentes e todos usam diversas técnicas de compressão em um grau ou outro.

Atualmente, o formato dominante para compressão de dados de áudio é o MP3. A popularidade do MP3 se deve principalmente ao fato de oferecer uma taxa de compressão mais forte do que outros formatos disponíveis na época. Outros formatos podem se mostrar mais eficientes no futuro, mas, por enquanto, o MP3 é o favorito geral. Em meados de 1999, o termo "MP3" era mais pesquisado do que qualquer outro termo, e continua forte até hoje. Vamos analisar os detalhes do formato MP3 um pouco mais de perto.

O Formato de Áudio MP3

MP3 é a abreviação de MPEG-2, arquivo de camada de áudio 3. MPEG é uma sigla para Moving Picture Experts Group (ou Grupo de Especialistas em Imagens em Movimento), um comitê internacional que desenvolve padrões para compressão digital de áudio e vídeo.

O MP3 emprega compressão com e sem perdas. Primeiro, ele analisa a propagação de frequência e a compara com modelos matemáticos da psicoacústica humana (o estudo da inter-relação entre o ouvido e o cérebro). Em seguida, descarta informações que não podem ser ouvidas por humanos. Finalmente, o fluxo de bits é compactado usando uma forma de codificação Huffman para obter compressão adicional.

Muitas ferramentas de software estão disponíveis na web para ajudar você a criar arquivos MP3. Essas ferramentas geralmente exigem que a gravação seja armazenada em algum outro formato comum, como WAV, antes que os dados sejam convertidos para o formato MP3, reduzindo significativamente o tamanho do arquivo.

Uma variedade de tocadores de MP3 interpreta e reproduz arquivos MP3. Um tocador de MP3 pode ser apenas um software para um computador existente ou pode ser um dispositivo de hardware dedicado que

armazena e reproduz os arquivos, como o popular iPod da Apple. A maioria dos tocadores de MP3 permite que os usuários organizem seus arquivos de diversas maneiras e exibam as informações e gráficos correspondentes durante a reprodução.

3.5 Representando Imagens e Gráficos

Imagens como fotografias e gráficos como desenhos lineares apresentam problemas comuns quando se trata de sua representação e compressão. Primeiro, examinamos a ideia geral de representação de cores e, em seguida, abordamos as diversas técnicas de digitalização e representação de informações visuais.

Representando Cores

A cor é a nossa percepção das várias frequências de luz que chegam às retinas dos nossos olhos. Nossas retinas possuem três tipos de células cone fotorreceptoras de cor que respondem a diferentes conjuntos de frequências. Essas categorias fotorreceptoras correspondem às cores vermelho, verde e azul. Todas as outras cores perceptíveis pelo olho humano podem ser formadas pela combinação de várias quantidades dessas três cores.

Em um computador, a cor é frequentemente expressa como um valor RGB (red-green-blue, ou vermelho-verde-azul), que são na verdade três números que indicam a contribuição relativa de cada uma dessas três cores primárias. Se cada número da tripla for dado em uma escala de 0 a 255, então 0 significa nenhuma contribuição daquela cor e 255 significa contribuição total daquela cor. Por exemplo, um valor RGB de (255, 255, 0) maximiza a contribuição do vermelho e do verde e minimiza a contribuição do azul, o que resulta em um amarelo brilhante. O conceito de valores RGB dá origem a um espaço de cores tridimensional.

A FIGURA 3.10, ao lado, mostra uma maneira de exibir esse espaço de cores.

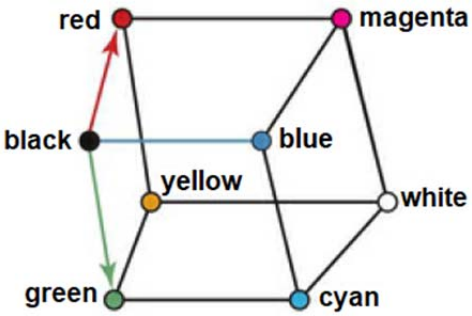


FIGURA 3.10 Um espaço de cores tridimensional.

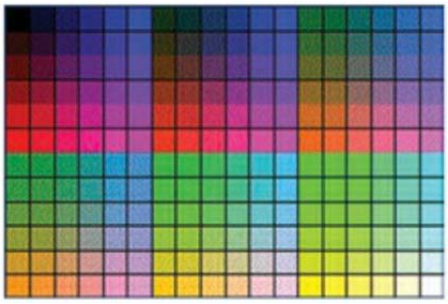
A quantidade de dados usada para representar uma cor é chamada de profundidade de cor. Geralmente, é expressa em termos do número de bits usados para representar a cor. High Color indica uma profundidade de cor de 16 bits. Com esse esquema, 5 bits são usados para cada número em um valor RGB e o bit extra às vezes é usado para representar a transparência. True Color indica uma profundidade de cor de 24 bits. Com esse esquema, cada número em um valor RGB recebe 8 bits, o que fornece o intervalo de 0 a 255 para cada um. Isso resulta na capacidade de representar mais de 16,7 milhões de cores únicas.

O quadro ao lado mostra alguns valores RGB de true color e as cores que eles representam.

Os valores de true color de 24 bits fornecem mais cores do que o olho humano consegue distinguir.

Valor RGB			
Red	**Green**	**Blue**	**Color**
0	0	0	black
255	255	255	white
255	255	0	yellow
255	130	255	pink
146	81	0	brown
157	95	82	purple
140	0	0	maroon

Além disso, os monitores que exibem cores são restritos a uma profundidade de cor específica. Em hardwares mais antigos, nos quais as cores do monitor são reduzidas a, digamos, 256 cores, qualquer cor especificada por um programa é mapeada para a cor mais próxima na paleta de cores que o hardware é capaz de exibir. A FIGURA 3.11 mostra uma paleta de cores tão restrita. Quando há diferenças significativas entre as cores desejadas e as cores que podem ser exibidas, os resultados geralmente são insatisfatórios. Felizmente, a maioria dos monitores modernos oferece gama de cores suficiente para reduzir significativamente esses problemas.


FIGURA 3.11 Uma paleta de cores restrita.

Imagens e Gráficos Digitalizados

Uma fotografia é uma representação analógica de uma imagem. Ela é contínua em toda a sua superfície, com tons de uma cor se misturando a outra. Digitalizar uma imagem é o ato de representá-la como uma coleção de pontos individuais chamados pixels, um termo que significa elementos de imagem. Cada pixel é

composto por uma única cor. O número de pixels usados para representar uma imagem é chamado de resolução. Se pixels suficientes forem usados (alta resolução) e, em seguida, apresentados na ordem correta, lado a lado, o olho humano pode ser enganado e pensar que está visualizando uma imagem contínua. A FIGURA 3.12 mostra uma imagem digitalizada, com uma pequena parte dela ampliada para mostrar os pixels individuais.

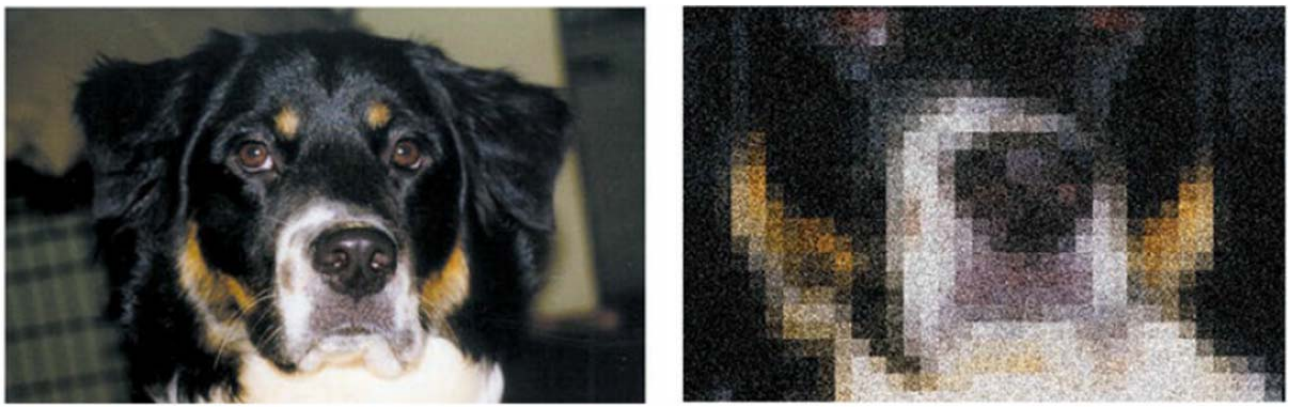


FIGURA 3.12 Uma imagem digitalizada composta por muitos pixels individuais.

Cortesia de Amy Rose

O armazenamento de informações da imagem pixel-a-pixel é chamado de formato raster-graphics. Vários formatos populares de arquivos raster-graphics estão em uso atualmente, incluindo bitmap (BMP), GIF e JPEG.

* Pixels: Pontos individuais usados para representar uma imagem; significa elementos de imagem.

* Resolução: O número de pixels usados para representar uma imagem.

* Formato raster-graphics: armazena informações da imagem pixel-a-pixel.

Um arquivo bitmap é uma das representações gráficas mais simples. Além de alguns detalhes administrativos, um arquivo bitmap contém os valores de cor dos pixels da imagem, da esquerda para a direita e de cima para baixo. Um arquivo bitmap suporta true color de 24 bits, embora geralmente a profundidade de cor possa ser especificada para reduzir o tamanho do arquivo. Esse arquivo pode ser compactado usando codificação run-length.

O GIF (Graphics Interchange Format, ou formato de intercambio gráfico), desenvolvido pela CompuServe em 1987, limita o número de cores disponíveis na imagem a 256. Ou seja, uma imagem GIF pode ser composta por apenas 256 cores, mas cada imagem GIF pode ser composta por um conjunto diferente de 256 cores. Essa técnica, chamada de cor indexada, resulta em tamanhos de arquivo menores porque há menos cores para referência. Se ainda menos cores forem necessárias, a profundidade de cor geralmente pode ser especificada com menos bits. Arquivos GIF são mais adequados para gráficos e imagens com poucas cores e, portanto, são considerados ideais para arte linear. Uma versão do formato GIF permite que pequenas animações sejam definidas armazenando uma série de imagens que um programa, como um navegador, exibe em sucessão.

O formato JPEG foi projetado para explorar a natureza dos nossos olhos. Humanos são mais sensíveis a mudanças graduais de brilho e cor à distância do que a mudanças rápidas. Portanto, os dados armazenados no formato JPEG calculam a média dos tons de cor em curtas distâncias. Este formato é considerado superior para imagens fotográficas coloridas. Um esquema de compactação bastante complicado pode reduzir significativamente o tamanho dos arquivos resultantes.

? Einstein descreve o telégrafo

"Veja bem, o telégrafo de fio é uma espécie de gato muito, muito comprido", explicou Albert Einstein. "Você puxa o rabo dele em Nova York e a cabeça dele mia em Los Angeles... E o rádio funciona exatamente da mesma maneira: você envia sinais aqui, eles os recebem lá. A única diferença é que não há gato." Como você acha que ele descreveria a Web?

O PNG (pronuncia-se "ping") significa Portable Network Graphics, ou Gráficos Portáteis de Rede. Os projetistas do formato PNG queriam aprimorar e, por fim, substituir o formato GIF. Imagens PNG geralmente conseguem uma compressão maior do que GIFs, oferecendo uma gama muito maior de profundidades de

cores. No entanto, imagens PNG não suportam animação e ainda não são tão amplamente suportadas quanto GIFs.

Representação Vetorial de Gráficos

Gráficos vetoriais são outra técnica para representar imagens. Em vez de atribuir cores a pixels como fazemos em arquivos raster-graphics, um formato de gráficos vetoriais descreve uma imagem em termos de linhas e formas geométricas. Um gráfico vetorial é uma série de comandos que descrevem a direção, espessura e cor de uma linha. Os tamanhos de arquivo produzidos com esses formatos tendem a ser pequenos, pois cada pixel não precisa ser contabilizado. A complexidade da imagem, como o número de itens na imagem, determina o tamanho do arquivo.

* Gráficos vetoriais: Representação de uma imagem em termos de linhas e formas.

Um arquivo raster-graphics, como um GIF, deve ser codificado várias vezes para levar em conta diferentes tamanhos e proporções. Em contraste, gráficos vetoriais podem ser redimensionados matematicamente, e essas alterações podem ser calculadas dinamicamente, conforme necessário.

No entanto, imagens de gráficos vetoriais não são boas para representar imagens do mundo real. Imagens JPEG são muito superiores nesse aspecto, mas imagens de gráficos vetoriais são boas para arte de linha e desenhos em estilo cartoon.

Um formato de gráficos vetoriais que foi muito popular usado na web foi o Flash. As imagens em Flash são armazenadas em formato binário e requerem um editor especial para serem criadas. Agora temos um novo formato de gráficos vetoriais, chamado Scalable Vector Graphics (SVG). O SVG é expresso em texto simples. O SVG é um padrão aberto e amplamente compatível com navegadores e dispositivos, tornando-se a escolha ideal para exibição de imagens vetoriais na internet. SVGs podem ser redimensionados sem perda de qualidade, o que é crucial para designs responsivos em diferentes tamanhos de tela.

3.6 Representando Vídeo

As informações de vídeo são um dos tipos de informação mais complexos de capturar, compactar e ainda obter um resultado que faça sentido para o olho humano. Os videoclipes contêm o equivalente a muitas imagens estáticas, cada uma das quais precisa ser compactada. A web está repleta de videoclipes com níveis de qualidade amplamente variados. Alguns dos problemas de qualidade dependem da sofisticação das técnicas de compactação de vídeo, chamadas de codecs de vídeo.

Codecs de Vídeo

Codec significa COmpressor/DECompressor, ou compressor/descompressor. Um codec de vídeo se refere aos métodos usados para reduzir o tamanho de um filme para que ele possa ser reproduzido em um computador ou em uma rede. Quase todos os codecs de vídeo usam compactação com perdas para minimizar as enormes quantidades de dados associadas ao vídeo. O objetivo, portanto, é não perder informações que afetam os sentidos do espectador.

* Codec de vídeo: Métodos usados para reduzir o tamanho de um filme.

A maioria dos codecs é orientada a blocos, o que significa que cada quadro de um vídeo é dividido em blocos retangulares. Os vários codecs diferem na forma como os blocos são codificados. Alguns codecs de vídeo são totalmente criados em software, enquanto outros exigem hardware especial.

Os codecs de vídeo empregam dois tipos de compressão: temporal e espacial. A compressão temporal procura diferenças entre quadros consecutivos. Se a maior parte de uma imagem em dois quadros não mudou, por que desperdiçaríamos espaço duplicando todas as informações semelhantes? Um keyframe (ou quadro-chave) é escolhido como base para comparar as diferenças, e toda a sua imagem é armazenada. Para imagens consecutivas, apenas as alterações (chamadas delta frames, ou quadros delta) são armazenadas. A compressão temporal é eficaz em vídeos que mudam pouco de quadro a quadro, como uma cena com pouco movimento.

* Compressão temporal: Técnica de compressão de filmes baseada nas diferenças entre quadros consecutivos.

A compressão espacial remove informações redundantes dentro de um quadro. Esse problema é essencialmente o mesmo que enfrentamos ao compactar imagens estáticas. A compressão espacial de vídeo

geralmente agrupa pixels em blocos (áreas retangulares) que têm a mesma cor, como uma parte de um céu azul claro. Em vez de armazenar cada pixel, a cor e as coordenadas da área são armazenadas. Essa ideia é semelhante à codificação de execução.

* Compressão espacial: Técnica de compressão de filmes baseada nas mesmas técnicas de compressão usadas para imagens estáticas.

Vários codecs de vídeo são populares atualmente, incluindo Sorenson, Cinepak, MPEG e Real Video. Os detalhes de como esses codecs representam e compactam vídeo estão além do escopo deste livro.

Bob Bemer



Bob Bemer tornou-se uma figura importante nos círculos da computação em 1945. Seu currículo parece uma lista das empresas de computação mais influentes do último meio século. Ele trabalhou para a Douglas Aircraft, RKO Radio Pictures, Rand Corporation, Lockheed Aircraft, Marquardt Aircraft, Lockheed Missiles and Space, IBM, Univac Division of Sperry Rand, Bull General Electric (Paris), GTE, Honeywell e, finalmente, sua própria empresa de software, a Bob Bemer Software.

A predominância de fabricantes de aeronaves no currículo de Bemer não é surpreendente, pois ele estudou matemática e obteve um Certificado em Engenharia Aeronáutica pelo Curtiss-Wright Technical Institute (1941). Nos primórdios da computação, os fabricantes de aeronaves foram pioneiros no uso de computadores na indústria.

Durante sua carreira, Bemer atuou no desenvolvimento de linguagens de programação. Ele desenvolveu o FORTRANSIT, um dos primeiros compiladores FORTRAN. Ele esteve ativamente envolvido no desenvolvimento da linguagem COBOL e da linguagem CODASYL, uma das primeiras abordagens para modelagem e gerenciamento de bancos de dados. Além disso, ele foi responsável por autorizar o financiamento para o desenvolvimento do SIMULA, uma linguagem de simulação que introduziu muitos recursos orientados a objetos.

Bemer também participou ativamente de comitês formados para trazer padrões universais para a nova indústria da computação. Foi representante dos EUA no Comitê de Vocabulário de Computação do IFIP, presidente da ISO/TC97/SC5 sobre Linguagens Comuns de Programação e presidente do Grupo de Estudo X3/SPARC sobre Processamento de Texto.

No entanto, Bemer é mais conhecido por seu trabalho com o código de computador ASCII, que é o código interno padrão para PCs de 8 bits atualmente. Logo no início, Bemer reconheceu que, se os computadores quisessem se comunicar entre si, precisariam de um código padrão para transmitir informações textuais. Bemer elaborou e publicou uma pesquisa com mais de 60 códigos de computador diferentes, demonstrando assim a necessidade de um código padrão. Ele criou o programa de trabalho para o comitê de padrões, forçou o código padrão dos EUA a corresponder ao código internacional, escreveu a maior parte dos artigos publicados sobre o código e pressionou por um registro formal de conjuntos de símbolos e controles alternativos ASCII para acomodar outras linguagens.

Talvez a contribuição mais importante de Bemer seja o conceito de caractere de escape. O caractere de escape alerta o sistema que processa os caracteres de que o(s) caractere(s) seguinte(s) altera(m) o significado padrão dos caracteres a seguir. Por exemplo, ESC (N) alerta o sistema de que os seguintes caracteres estão no cirílico equivalente ao ASCII.

A primeira versão de um código de 16 bits chamado Unicode foi publicada em outubro de 1991. Dois fatores impulsionaram a necessidade de um código ampliado: a arquitetura de computadores de 16 bits estava se tornando popular, e a expansão da internet e da World Wide Web impulsionou a necessidade de um código que pudesse incluir diretamente os alfabetos do mundo. O ASCII, no entanto, não desapareceu; ele continua sendo um subconjunto do Unicode.

Em maio de 2003, Bemer recebeu o Prêmio de Pioneiro em Computação da IEEE Computer Society por atender às necessidades mundiais de conjuntos de caracteres variantes e outros símbolos, via ASCII, conjuntos alternativos ASCII e sequências de escape".

Bob Bemer faleceu em 22 de junho de 2004, em sua casa no Lago Possum Kingdom, no Texas.[2]

RESUMO

Computadores são dispositivos multimídia que manipulam dados que variam em formato, desde números a gráficos e vídeo. Como um computador só pode manipular valores binários, todas as formas de dados devem ser representadas em formato binário. Os dados são classificados como contínuos (analógicos) ou discretos (digitais). Valores inteiros são representados por seu equivalente binário usando uma das várias técnicas para representar números negativos, como uma magnitude com sinal ou complemento de dois. Números reais são representados por uma tripla composta pelo sinal, os dígitos do número e um expoente que especifica o ponto de base.

Um conjunto de caracteres é uma lista de caracteres alfanuméricos e os códigos que representam cada caractere. O conjunto de caracteres mais comumente usado atualmente é o Unicode (16 bits para cada caractere), que tem o ASCII como subconjunto. O conjunto ASCII de 8 bits é suficiente para o inglês, mas não para outros (ou vários) idiomas. Existem várias maneiras de compactar texto para que ocupe menos espaço para armazená-lo ou menos tempo para transmiti-lo de uma máquina para outra.

As informações de áudio são representadas como ondas sonoras digitalizadas. A cor é representada por três valores que representam as contribuições de vermelho, azul e verde, respectivamente. Duas técnicas básicas são usadas para representar imagens, bitmaps e gráficos vetoriais. O vídeo é dividido em uma série de imagens estáticas, cada uma delas representada como uma imagem.

EXERCÍCIOS

Para os Exercícios 1 a 20, marque as respostas como verdadeiras ou falsas, da seguinte forma:

A. Verdadeiro B. Falso

1. Compressão sem perdas significa que os dados podem ser recuperados sem perder nenhuma informação original.
2. Um computador representa informações em formato analógico.
3. Um computador deve usar o sistema numérico binário para representar informações.
4. Um sinal digital representa um de dois valores em qualquer ponto no tempo.
5. Quatro bits podem ser usados para representar 32 coisas únicas.
6. A representação de números com sinal de magnitude tem duas representações para zero.
7. O estouro ocorre quando o valor que calculamos não cabe no número de bits que alocamos para o resultado.
8. No conjunto de caracteres ASCII, não há distinção entre letras maiúsculas e minúsculas.
9. O conjunto de caracteres Unicode inclui todos os caracteres do conjunto de caracteres ASCII.
10. A codificação por palavras-chave substitui palavras usadas com frequência por um único caractere.
11. A codificação run-length é muito boa para compactar textos em inglês.
12. A codificação Huffman usa strings binárias de comprimento variável para representar caracteres.
13. Um sinal de áudio é digitalizado por amostragem em intervalos regulares.
14. Um CD armazena informações de áudio em formato binário.
15. O formato de áudio MP3 descarta informações que não podem ser ouvidas por humanos.
16. Um valor RGB representa uma cor usando três valores numéricos.
17. A cor indexada aumenta o número de cores que podem ser usadas em uma imagem e, portanto, aumenta o tamanho do arquivo.
18. Bitmap, GIF e JPEG são exemplos de formatos de gráficos raster.
19. Gráficos vetoriais representam imagens em termos de linhas e formas geométricas.
20. Um quadro-chave é usado na compressão temporal para representar as mudanças de um quadro para outro.

Para os Exercícios 21 a 26, escolha o termo correto na lista a seguir.

A. Representação de magnitude com sinal.
B. Ponto base.
C. Frequência de uso.
D. Amostragem.
E. Analógico.
F. Digital.

21. Dados _____ são uma representação contínua de informações.
22. A representação para números que você usa desde o ensino fundamental é chamada de _____.
23. Se a base numérica for diferente de 10, chamamos o ponto decimal de _____.
24. Dados _____ são uma representação discreta de informações.
25. Os códigos de Huffman são criados com base no _____ do caractere.
26. Um sinal de áudio é digitalizado _____ seu valor em intervalos regulares.

Os exercícios 27 a 79 são problemas ou perguntas de resposta curta.

27. Por que a compressão de dados é um tópico importante hoje em dia?
28. Qual é a diferença entre compressão de dados sem perdas e com perdas?
29. Por que os computadores têm dificuldade com informações analógicas?
30. Um relógio com ponteiro de segundos oscilante é um dispositivo analógico ou digital? Explique.
31. O que significa digitalizar algo?
32. O que é modulação por código de pulso?
33. Quantas coisas podem ser representadas com
    a) Quatro bits? b) Cinco bits? c) Seis bits? d) Sete bits?
34. Embora você tenha calculado operações aritméticas simples desde a segunda série, faça o seguinte teste rápido para confirmar que você entende completamente operações com números inteiros com sinal. Avalie as seguintes expressões em que W é 17, X é 28, Y é –29 e Z é –13.
    a) X + Y b) X + W c) Z + W d) Y + Z e) W – Z f) X – W g) Y – W h) Z – Y
35. Use a reta numérica de base 10 para provar as soluções das seguintes operações, onde A é 5 e B é –7.
    a) A + B b) A – B c) B + A d) B – A
36. Dado um esquema numérico de tamanho fixo onde k na fórmula para o complemento de dez é 6 (veja a página 63), responda às seguintes perguntas.
    a) Quantos números inteiros positivos podem ser representados?
    b) Quantos números inteiros negativos podem ser representados?
    c) Desenhe a reta numérica mostrando os três menores e maiores números positivos, os três menores e maiores números negativos e zero.
37. Use a reta numérica do Exercício 36(c) para calcular as seguintes expressões, onde A é –499999 e B é 3.
    a) A + B b) A – B c) B + A d) B – A
38. Use a fórmula do complemento de dez para calcular os seguintes números no esquema descrito na página 63.
    a) 35768 b) –35768 c) –444455 d) –123456
39. Ao calcular o complemento de dez no Exercício 38, você teve dificuldade em tomar emprestado de tantos zeros? Esses cálculos são propensos a erros. Há um truque que você pode usar para tornar o cálculo mais fácil e, portanto, menos propenso a erros: Subtraia de todos os 9s e, em seguida, adicione 1. Um número subtraído de todos os 9s é chamado de complemento de nove do número.
    a) Prove que o complemento de nove de um número mais um é igual ao complemento de dez do mesmo número.
    b) Use o complemento de nove mais um para calcular os valores no Exercício 38(b), (c) e (d).
    c) Qual você achou mais fácil de usar: o cálculo direto do complemento de dez ou o complemento de nove mais um? Justifique sua resposta.
40. Avalie as seguintes expressões, onde A é 11111110 e B é 00000010, usando o complemento de dois.
    a) A + B b) A – B c) B – A d) –B e) – (–A)

41. O complemento de dois de um número é sempre um número negativo? Explique.
42. Elabore um sistema numérico baseado na base 11.
    a) Desenhe a reta numérica.
    b) Mostre exemplos de adição e subtração.
    c) Desenvolva uma representação de números negativos com base no complemento de onze.
43. Converta as regras de subtração em um sistema de grandeza com sinal para o formato do algoritmo
44. Converta os seguintes números reais para binário (cinco casas binárias).
    a) 0,50 b) 0,26 c) 0,10
45. Converta os seguintes números reais para octal (cinco casas octais).
    a) 0,50 b) 0,26 c) 0,10
46. Valores fracionários podem ser convertidos visualmente entre octal e binário e vice-versa? Explique.
47. Quantos bits seriam necessários para representar um conjunto de caracteres contendo 45 caracteres? Por quê?
48. Como o número decimal 175,23 pode ser representado como um sinal, mantissa e expoente?
49. Qual é a principal diferença entre os conjuntos de caracteres ASCII e Unicode?
50. Crie uma tabela de codificação de palavras-chave que contenha algumas palavras simples. Reescreva um parágrafo de sua escolha usando este esquema de codificação. Calcule a taxa de compressão obtida.
51. Como a seguinte sequência de caracteres seria representada usando codificação run-length? Qual é a taxa de compressão? AAAABBBCCCCCCCCDDDD oi aí EEEEEEEEEFF
52. O que o código *X5*A9 representa usando codificação run-length?
53. Dada a seguinte tabela de codificação de Huffman, decifre as sequências de bits a seguir.

Código de Huffman	Caractere
00	A
11	E
010	T
0110	C
0111	L
1000	S
1011	R
10010	O
10011	I
101000	N
101001	F
101010	H
101011	D

    a) 1101110001011
    b) 011010101010010101111000
    c) 10100100101000010001000010 1 00110110
    d) 10100010010101000100011101 0 00100011
54. Como os humanos percebem o som?
55. Um alto-falante estéreo é um dispositivo analógico ou digital? Explique.
56. O que é um valor RGB?
57. O que a profundidade de cor indica?
58. Como a resolução de pixels afeta o impacto visual de uma imagem?
59. Explique a compressão temporal de vídeo.
60. Descreva uma situação em que a compressão espacial de vídeo seria eficaz.
61. Defina amostragem em relação à digitalização de ondas sonoras.
62. O que produz melhor qualidade de som, taxas de amostragem mais altas ou taxas de amostragem mais baixas?
63. Qual é a taxa de amostragem por segundo suficiente para criar uma reprodução sonora razoável?
64. Os discos de vinil e os CDs gravam som da mesma forma?

65. O que significa um valor RGB de (130, 0, 255)?
66. Qual cor um valor RGB de (255, 255, 255) representa?
67. O que é resolução?
68. O formato GIF usa qual técnica?
69. Para que os arquivos GIF são mais adequados?
70. Quais são as semelhanças entre os vários codecs de vídeo?
71. Quais são as diferenças entre os vários codecs de vídeo?
72. Cite dois tipos de compressão de vídeo.
73. Como chamamos a percepção das diferentes frequências de luz que atingem a retina dos nossos olhos?
74. Qual é o melhor formato para imagens fotográficas coloridas?
75. Como são chamadas as técnicas que reduzem o tamanho dos filmes?
76. Qual é a técnica na qual um aplicativo suporta apenas um certo número de cores específicas, criando uma paleta para escolher?
77. Qual é o formato que descreve uma imagem em termos de linhas e formas geométricas?
78. Qual formato armazena informações pixel a pixel?
79. Qual é a diferença entre high color e true color?

QUESTÕES PARA REFLEXÃO

1. Quais são algumas vantagens de usar um conjunto de caracteres comum (padronizado)? Quais são algumas desvantagens?
2. Ao converter números inteiros de uma base para outra, dividimos pela nova base. Ao converter partes fracionárias de uma base para outra, multiplicamos pela nova base. A notação posicional pode ser usada para explicar esses algoritmos?
3. A tecnologia está mudando rapidamente. Quais mudanças ocorreram na compactação de dados desde que este livro foi escrito?
4. Onde está Edward Snowden agora?
5. Como você acha que a história o chamará? Já acharam? Como?